

Mini RFC: Pacing & Persistent Congestion

April 5, 2022

Note: This is a simplified and annotated summary of the QUIC-related IETF RFCs. It's part of a course project at Tsinghua University and not suitable for any other purposes.

We reorder paragraphs in QUIC RFCs and omit many details to improve its readability for beginners.

WARNING: DO NOT distribute this document. Modification of extracts from IETF RFCs is not granted by IETF as original authors retain copyrights of RFCs. This document is for educational purposes only.

1 Pacing

A sender **SHOULD** pace sending of all in-flight packets based on input from the congestion controller.

Sending multiple packets into the network without any delay between them creates a packet burst that might cause short-term congestion and losses. Senders **MUST** either use pacing or limit such bursts. Senders **SHOULD** limit bursts to the initial congestion window. A sender with knowledge that the network path to the receiver can absorb larger bursts **MAY** use a higher limit.

Bursty packets easily overflow short queues on network devices and get dropped. Such lost packets lead to throughput downgrades even when the link is not fully utilized.

An implementation should take care to architect its congestion controller to work well with a pacer. For instance, a pacer might wrap the congestion controller and control the availability of the congestion window, or a pacer might pace out packets handed to it by the congestion controller.

Timely delivery of ACK frames is important for efficient loss recovery. To avoid delaying their delivery to the peer, packets containing only ACK frames **SHOULD** therefore not be paced.

Endpoints can implement pacing as they choose. A perfectly paced sender spreads packets exactly evenly over time. For a window-based congestion controller, such as the one in this document, that rate can be computed by averaging the congestion window over the RTT. Expressed as a rate in units of bytes per time, where `congestion_window` is in bytes:

$$\text{rate} = N * \text{congestion_window} / \text{smoothed_rtt}$$

Or expressed as an inter-packet interval in units of time:

$$\text{interval} = (\text{smoothed_rtt} * \text{packet_size} / \text{congestion_window}) / N$$

Using a value for N that is small, but at least 1 (for example, 1.25) ensures that variations in RTT do not result in underutilization of the congestion window.

Practical considerations, such as packetization, scheduling delays, and computational efficiency, can cause a sender to deviate from this rate over time periods that are much shorter than an RTT.

One possible implementation strategy for pacing uses a leaky bucket algorithm, where the capacity of the "bucket" is limited to the maximum burst size and the rate the "bucket" fills is determined by the above function.

2 Persistent Congestion

When a sender establishes loss of all packets sent over a long enough duration, the network is considered to be experiencing persistent congestion.

2.1 Duration

The persistent congestion duration is computed as follows:

$$(\text{smoothed_rtt} + \max(4 * \text{rttvar}, \text{kGranularity}) + \text{max_ack_delay}) * \text{kPersistentCongestionThreshold}$$

Unlike the PTO computation in [mini RFC: PTO](#), this duration includes the `max_ack_delay` irrespective of the packet number spaces in which losses are established.

This duration allows a sender to send as many packets before establishing persistent congestion, including some in response to PTO expiration, as TCP does with Tail Loss Probes [\[RFC 8985\]](#) and an RTO [\[RFC 5681\]](#).

Larger values of `kPersistentCongestionThreshold` cause the sender to become less responsive to persistent congestion in the network, which can result in aggressive sending into a congested network. Too small a value can result in a sender declaring persistent congestion unnecessarily, resulting in reduced throughput for the sender.

The RECOMMENDED value for `kPersistentCongestionThreshold` is 3, which results in behavior that is approximately equivalent to a TCP sender declaring an RTO after two Tail Loss Probes.

This design does not use consecutive PTO events to establish persistent congestion, since application patterns impact PTO expiration. For example, a sender that sends small amounts of data with silence periods between them restarts the PTO timer every time it sends, potentially preventing the PTO timer from expiring for a long period of time, even when no acknowledgments are being received. The use of a duration enables a sender to establish persistent congestion without depending on PTO expiration.

2.2 Establishing Persistent Congestion

A sender establishes persistent congestion after the receipt of an acknowledgment if two packets that are ack-eliciting are declared lost, and:

- across all packet number spaces, none of the packets sent between the send times of these two packets are acknowledged;
- the duration between the send times of these two packets exceeds the persistent congestion duration;
- a prior RTT sample existed when these two packets were sent.

These two packets MUST be ack-eliciting, since a receiver is required to acknowledge only ack-eliciting packets within its maximum acknowledgment delay.

The persistent congestion period SHOULD NOT start until there is at least one RTT sample. Before the first RTT sample, a sender arms its PTO timer based on the initial RTT, which could be substantially larger than the actual RTT. Requiring a prior RTT sample prevents a sender from establishing persistent congestion with potentially too few probes.

Since network congestion is not affected by packet number spaces, persistent congestion SHOULD consider packets sent across packet number spaces. A sender that does not have state for all packet number spaces or an implementation that cannot compare send times across packet number spaces MAY use state for just the packet number space that was acknowledged. This might result in erroneously declaring persistent congestion, but it will not lead to a failure to detect persistent congestion.

When persistent congestion is declared, the sender's congestion window MUST be reduced to the minimum congestion window (`kMinimumWindow`), similar to a TCP sender's response on an RTO [RFC 5681].

and delay non-ack-eliciting packets for an arbitrarily long time.